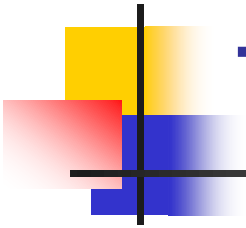
Workshops: Image convolution and Filtring

Pro Jamshid Dehmeshki

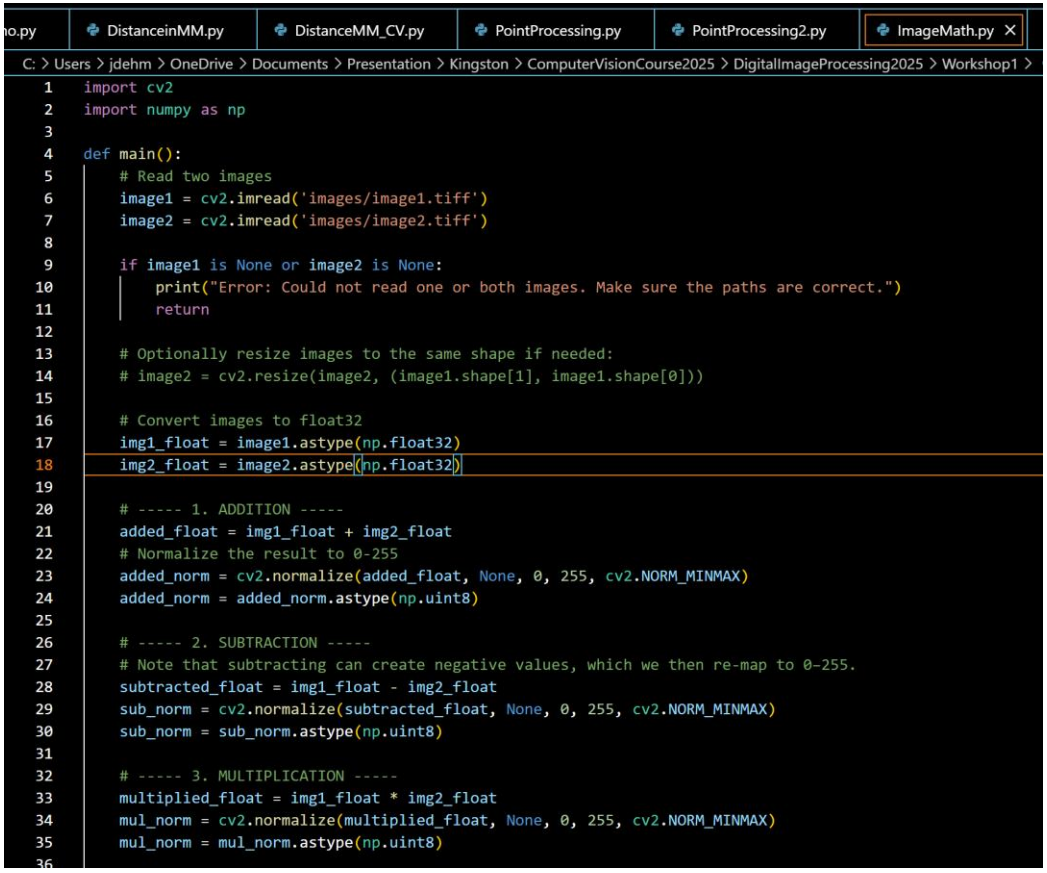
Image Arithmetic

- **Description**
  - Image arithmetic refers to performing operations such as addition, subtraction, or multiplication on images (often pixel-wise).
  - Typical use cases include blending two images, adjusting brightness, or creating mask effects.



Task 1:

- Load two images (e.g. gray level)
- Apply pixel wise “addition, subtraction, or multiplication on images”
- Display images


DistanceinMM.py
DistanceMM_CV.py
PointProcessing.py
PointProcessing2.py
ImageMath.py
C: > Users > jdehm > OneDrive > Documents > Presentation > Kingston > ComputerVisionCourse2025 > DigitalImageProcessing2025 > Workshop1 >
import cv2
import numpy as np
def main():
Read two images
image1 = cv2.imread('images/image1.tiff')
image2 = cv2.imread('images/image2.tiff')
if image1 is None or image2 is None:
print("Error: Could not read one or both images. Make sure the paths are correct.")
return
Optionally resize images to the same shape if needed:
image2 = cv2.resize(image2, (image1.shape[1], image1.shape[0]))
Convert images to float32
img1_float = image1.astype(np.float32)
img2_float = image2.astype(np.float32)
----- 1. ADDITION -----
added_float = img1_float + img2_float
Normalize the result to 0-255
added_norm = cv2.normalize(added_float, None, 0, 255, cv2.NORM_MINMAX)
added_norm = added_norm.astype(np.uint8)
----- 2. SUBTRACTION -----
Note that subtracting can create negative values, which we then re-map to 0-255.
subtracted_float = img1_float - img2_float
sub_norm = cv2.normalize(subtracted_float, None, 0, 255, cv2.NORM_MINMAX)
sub_norm = sub_norm.astype(np.uint8)
----- 3. MULTIPLICATION -----
multiplied_float = img1_float * img2_float
mul_norm = cv2.normalize(multiplied_float, None, 0, 255, cv2.NORM_MINMAX)
mul_norm = mul_norm.astype(np.uint8)


Display results
cv2.imshow("Image 1", image1)
cv2.imshow("Image 2", image2)
cv2.imshow("Added (Normalized)", added_norm)
cv2.imshow("Subtracted (Normalized)", sub_norm)
cv2.imshow("Multiplied (Normalized)", mul_norm)
cv2.waitKey(0)
cv2.destroyAllWindows()
if __name__ == "__main__":
main()

Results

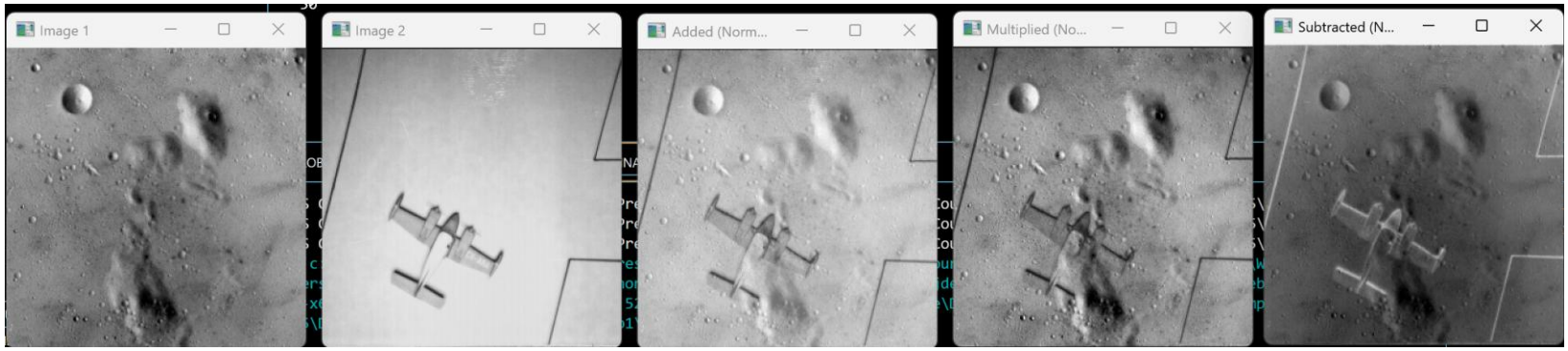
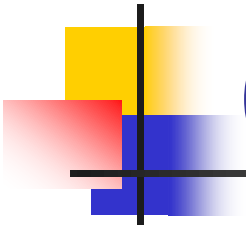


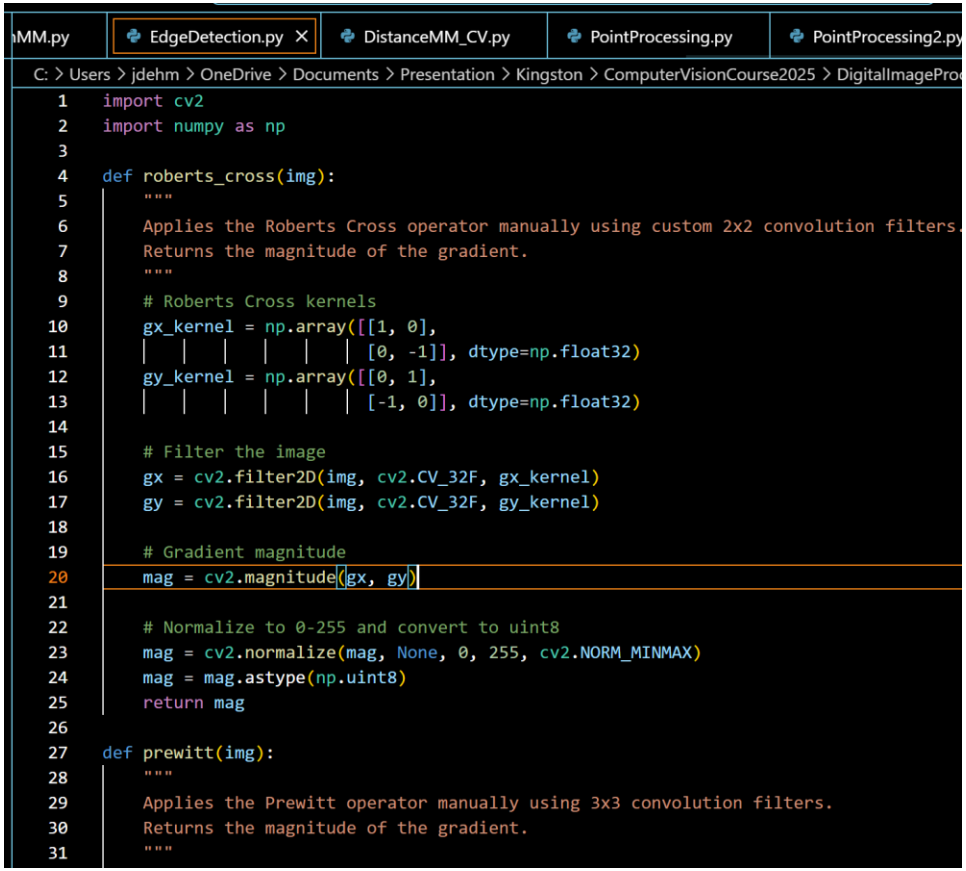
Image Spatial Filtering (Convolution)



- Spatial filtering (linear filtering) involves convolving an image with a kernel (also called a filter mask) to achieve effects such as blurring (low-pass filtering), sharpening (high-pass filtering), and more.
- Convolution is done by sliding the kernel across the image and computing a weighted sum of the pixel values covered by the kernel.

Task 2: Edge detection filtering

- Load an image convert it to grayscale and apply Sobel, Roberts, Prewitt, Scharr, and Laplacian—using both OpenCV and custom convolution kernels.
- These methods work best on grayscale images, so make sure your input image is in grayscale (or convert it).


MM.py
EdgeDetection.py
DistanceMM_CV.py
PointProcessing.py
PointProcessing2.py
C: > Users > jdehm > OneDrive > Documents > Presentation > Kingston > ComputerVisionCourse2025 > DigitalImagePro
import cv2
import numpy as np
def roberts_cross(img):
"""
Applies the Roberts Cross operator manually using custom 2x2 convolution filters.
Returns the magnitude of the gradient.
"""
Roberts Cross kernels
gx_kernel = np.array([[1, 0],
[0, -1]], dtype=np.float32)
gy_kernel = np.array([[0, 1],
[-1, 0]], dtype=np.float32)
Filter the image
gx = cv2.filter2D(img, cv2.CV_32F, gx_kernel)
gy = cv2.filter2D(img, cv2.CV_32F, gy_kernel)
Gradient magnitude
mag = cv2.magnitude(gx, gy)
Normalize to 0-255 and convert to uint8
mag = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
mag = mag.astype(np.uint8)
return mag
def prewitt(img):
"""
Applies the Prewitt operator manually using 3x3 convolution filters.
Returns the magnitude of the gradient.
"""

```python

def prewitt(img):
    """
    Applies the Prewitt operator manually using 3x3 convolution filters.
    Returns the magnitude of the gradient.
    """
    # Prewitt kernels
    gx_kernel = np.array([[ -1, 0, 1],
                          [ -1, 0, 1],
                          [ -1, 0, 1]], dtype=np.float32)

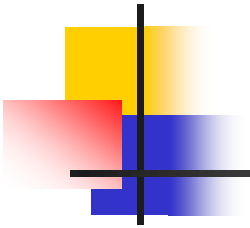
    gy_kernel = np.array([[  1,  1,  1],
                          [  0,  0,  0],
                          [ -1, -1, -1]], dtype=np.float32)

    # Filter the image
    gx = cv2.filter2D(img, cv2.CV_32F, gx_kernel)
    gy = cv2.filter2D(img, cv2.CV_32F, gy_kernel)

    # Gradient magnitude
    mag = cv2.magnitude(gx, gy)

    # Normalize to 0-255 and convert to uint8
    mag = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
    mag = mag.astype(np.uint8)
    return mag

```



```python
def main():
    # 1. Read/Load the image in Grayscale
    image_path = 'images/image3.tiff'
    img_gray = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if img_gray is None:
        print(f"Error reading image: {image_path}")
        return

    # 2. Apply Sobel Edge Detector (using OpenCV)
    # cv2.Sobel(src, ddepth, dx, dy, ksize)
    sobelx = cv2.Sobel(img_gray, cv2.CV_32F, 1, 0, ksize=3)  # Gx
    sobely = cv2.Sobel(img_gray, cv2.CV_32F, 0, 1, ksize=3)  # Gy
    sobel_mag = cv2.magnitude(sobelx, sobely)
    sobel_mag = cv2.normalize(sobel_mag, None, 0, 255, cv2.NORM_MINMAX)
    sobel_mag = sobel_mag.astype(np.uint8)

    # 3. Apply Scharr Edge Detector (using OpenCV)
    # Scharr is like Sobel but with a different kernel (ksize = -1).
    scharrx = cv2.Scharr(img_gray, cv2.CV_32F, 1, 0)
    scharry = cv2.Scharr(img_gray, cv2.CV_32F, 0, 1)
    scharr_mag = cv2.magnitude(scharrx, scharry)
    scharr_mag = cv2.normalize(scharr_mag, None, 0, 255, cv2.NORM_MINMAX)
    scharr_mag = scharr_mag.astype(np.uint8)

    # 4. Apply Laplacian (using OpenCV)
    # Laplacian detects edges by approximating the second derivative.
    laplacian = cv2.Laplacian(img_gray, cv2.CV_32F, ksize=3)
    laplacian = cv2.convertScaleAbs(laplacian)  # Convert to uint8 with absolute value

    # 5. Apply Roberts Cross Operator (Manual)
    roberts_img = roberts_cross(img_gray)

    # 6. Apply Prewitt Operator (Manual)
    prewitt_img = prewitt(img_gray)
```

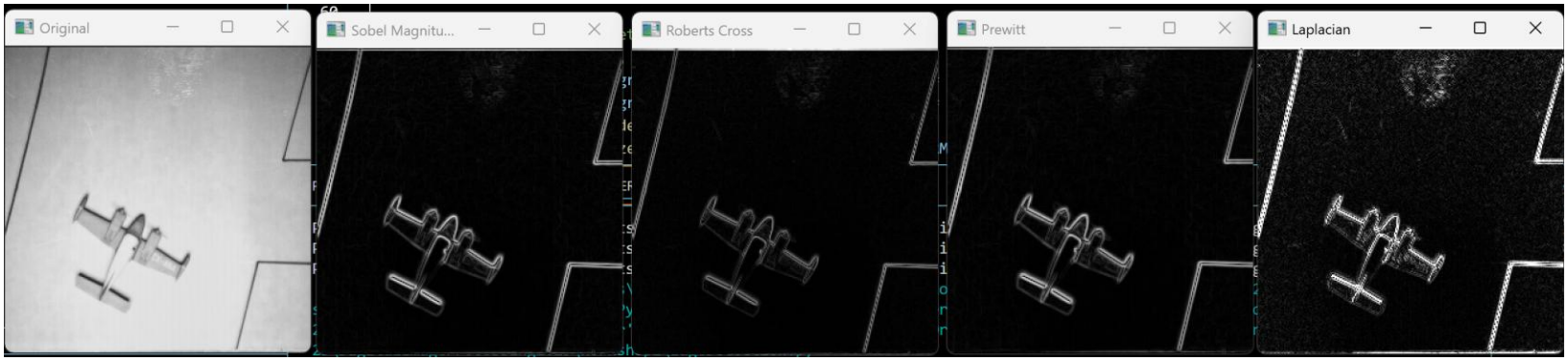
```
    # 6. Apply Prewitt Operator (Manual)
    prewitt_img = prewitt(img_gray)

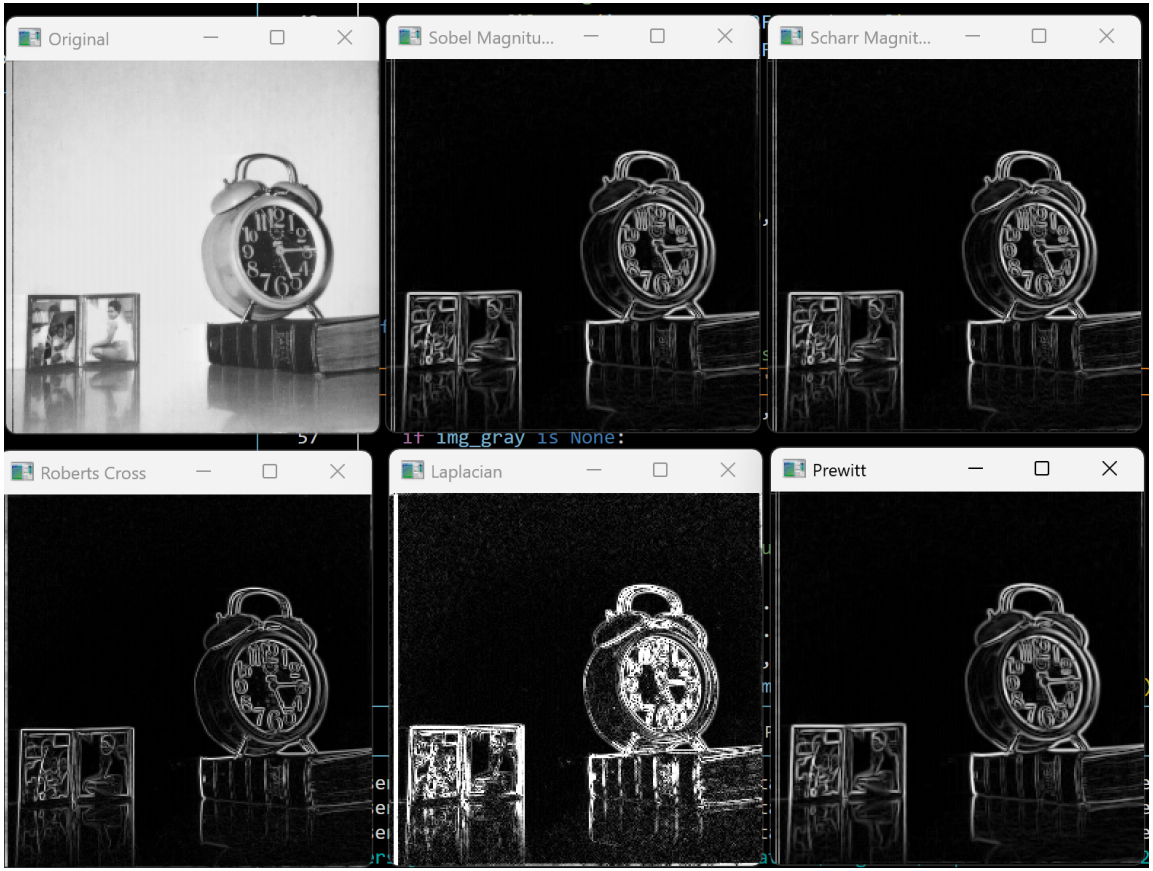
    # Display all results
    cv2.imshow("Original", img_gray)
    cv2.imshow("Sobel Magnitude", sobel_mag)
    cv2.imshow("Scharr Magnitude", scharr_mag)
    cv2.imshow("Laplacian", laplacian)
    cv2.imshow("Roberts Cross", roberts_img)
    cv2.imshow("Prewitt", prewitt_img)

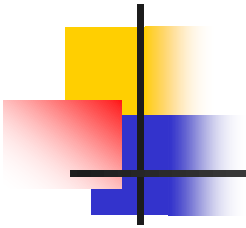
    cv2.waitKey(0)
    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```

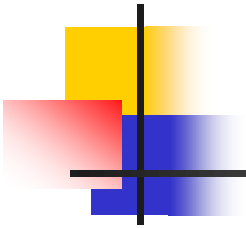

Original
Sobel Magnitu...
Roberts Cross
Prewitt
Laplacian


Original
Sobel Magnitu...
Scharr Magnit...
if img_gray is None:
Roberts Cross
Laplacian
Prewitt



Edge detection

- **Sobel (OpenCV):**
  - **Uses two 3x3 kernels**
  - **Relatively more resistant to noise compared to smaller kernels (like Roberts).**
  - **cv2.Sobel(img, depth, dx, dy, ksize) calculates the derivative in x-direction (dx=1, dy=0) or in y-direction (dx=0, dy=1). We take the magnitude of both.**
- **Scharr (OpenCV):**
  - **A specific case of Sobel with a larger kernel size (the parameter ksize=-1 in OpenCV).**
  - **Often gives better edge detection results in diagonal directions or near corners.**



Edge detection

- **Laplacian (OpenCV):**
  - **Approximates the second derivative. Edges appear where there is rapid change in intensity.**
  - **cv2.Laplacian(img, depth, ksize=3) is commonly used.**
- **Roberts Cross (Manual):**
  - **One of the oldest operators, uses a 2x2 kernel for Gx and Gy.**
  - **More sensitive to noise due to its small kernel size.**
- **Prewitt (Manual):**
  - **Uses two 3x3 kernels (one for horizontal edges, one for vertical edges).**
  - **Similar to Sobel, but with slightly different coefficients (Sobel has weighting of 2 in the center row/column, Prewitt does not).**

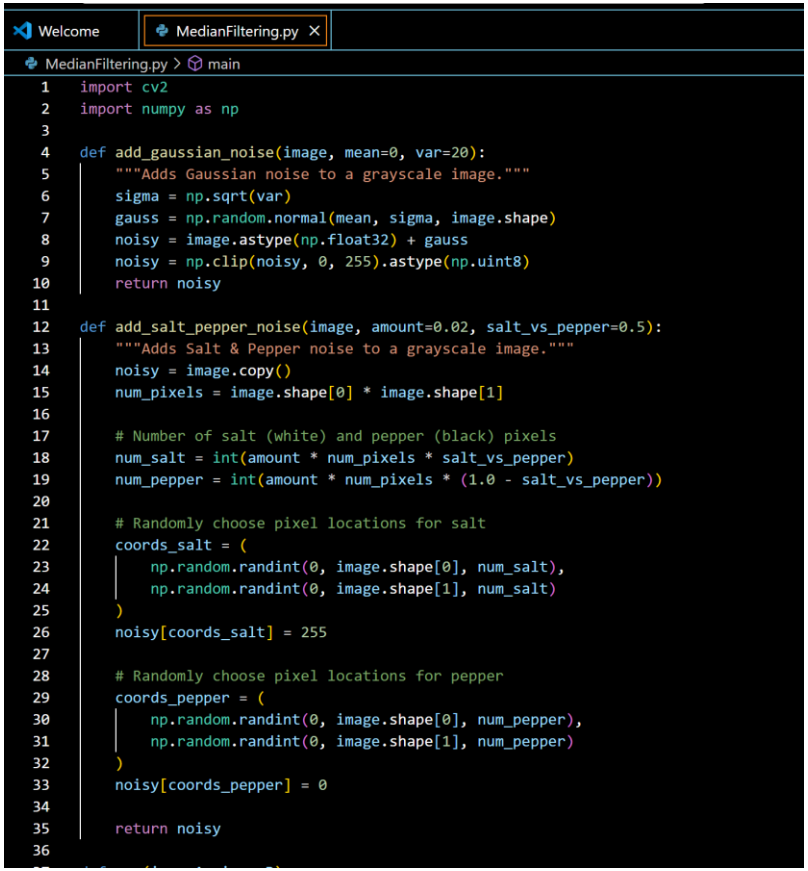
Non-Linear Filtering (Median)

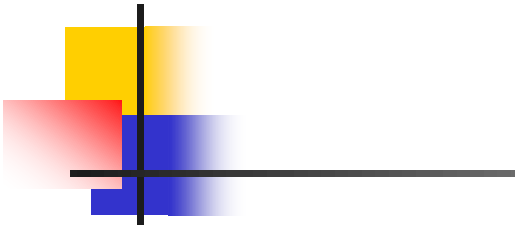
- **Description**

Median filtering sorts the pixel values within a neighborhood (e.g., 3×3 window) and takes the median value. This filter is especially effective for removing "salt-and-pepper" noise.

Task 3: Filtering

1. Load a Grayscale Image
   Reads the image in grayscale mode (cv2.IMREAD_GRAYSCALE).
2. Add Noise
   1. Gaussian Noise: Randomly perturbs pixel values following a normal distribution (with mean=0 and variance=20).
   2. Salt & Pepper Noise: Randomly turns a fraction of pixels white or black (amount=0.02 means 2% of pixels changed).
3. Filter
   1. Average Filter (cv2.blur) with a 5×5 kernel on the Gaussian-noisy image.
   2. Median Filter (cv2.medianBlur) with a 5×5 neighborhood on the Salt & Pepper-noisy image.
4. Compute MSE
   1. Mean Squared Error (MSE) quantifies how different each noisy or filtered image is from the original. Lower MSE indicates less distortion.
5. Display Images Individually


Welcome
MedianFiltering.py
MedianFiltering.py > main
import cv2
import numpy as np
def add_gaussian_noise(image, mean=0, var=20):
"""Adds Gaussian noise to a grayscale image."""
sigma = np.sqrt(var)
gauss = np.random.normal(mean, sigma, image.shape)
noisy = image.astype(np.float32) + gauss
noisy = np.clip(noisy, 0, 255).astype(np.uint8)
return noisy
def add_salt_pepper_noise(image, amount=0.02, salt_vs_pepper=0.5):
"""Adds Salt & Pepper noise to a grayscale image."""
noisy = image.copy()
num_pixels = image.shape[0] * image.shape[1]
Number of salt (white) and pepper (black) pixels
num_salt = int(amount * num_pixels * salt_vs_pepper)
num_pepper = int(amount * num_pixels * (1.0 - salt_vs_pepper))
Randomly choose pixel locations for salt
coords_salt = (
np.random.randint(0, image.shape[0], num_salt),
np.random.randint(0, image.shape[1], num_salt)
)
noisy[coords_salt] = 255
Randomly choose pixel locations for pepper
coords_pepper = (
np.random.randint(0, image.shape[0], num_pepper),
np.random.randint(0, image.shape[1], num_pepper)
)
noisy[coords_pepper] = 0
return noisy



```python

def mse(imageA, imageB):
    """Computes the Mean Squared Error between two images."""
    return np.mean((imageA.astype(np.float32) - imageB.astype(np.float32)) ** 2)

def main():
    # 1. Load an image in grayscale
    image_path = 'images/Peper.tiff'
    original = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if original is None:
        print(f"Error: Could not read {image_path}")
        return

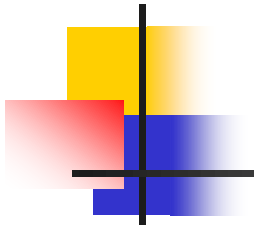
    # 2. Generate two noisy versions: Gaussian, Salt & Pepper
    noisy_gaussian = add_gaussian_noise(original, mean=0, var=100)
    noisy_sp = add_salt_pepper_noise(original, amount=0.02, salt_vs_pepper=0.5)

    # 3. Apply filters
    #    - Average filter on Gaussian noise
    avg_gaussian = cv2.blur(noisy_gaussian, (5,5))
    #    - Median filter on Salt & Pepper noise
    med_sp = cv2.medianBlur(noisy_sp, 5)

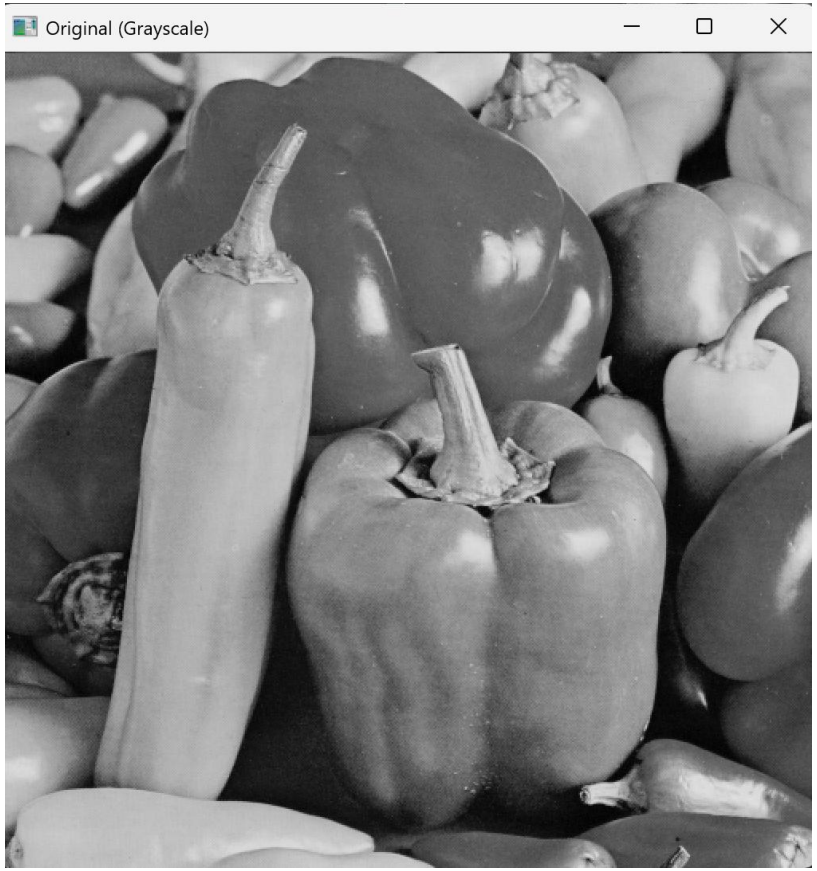
    # 4. Calculate MSE for each version vs. original
    mse_gaussian_noisy = mse(original, noisy_gaussian)
    mse_sp_noisy       = mse(original, noisy_sp)
    mse_avg_gaussian   = mse(original, avg_gaussian)
    mse_med_sp         = mse(original, med_sp)

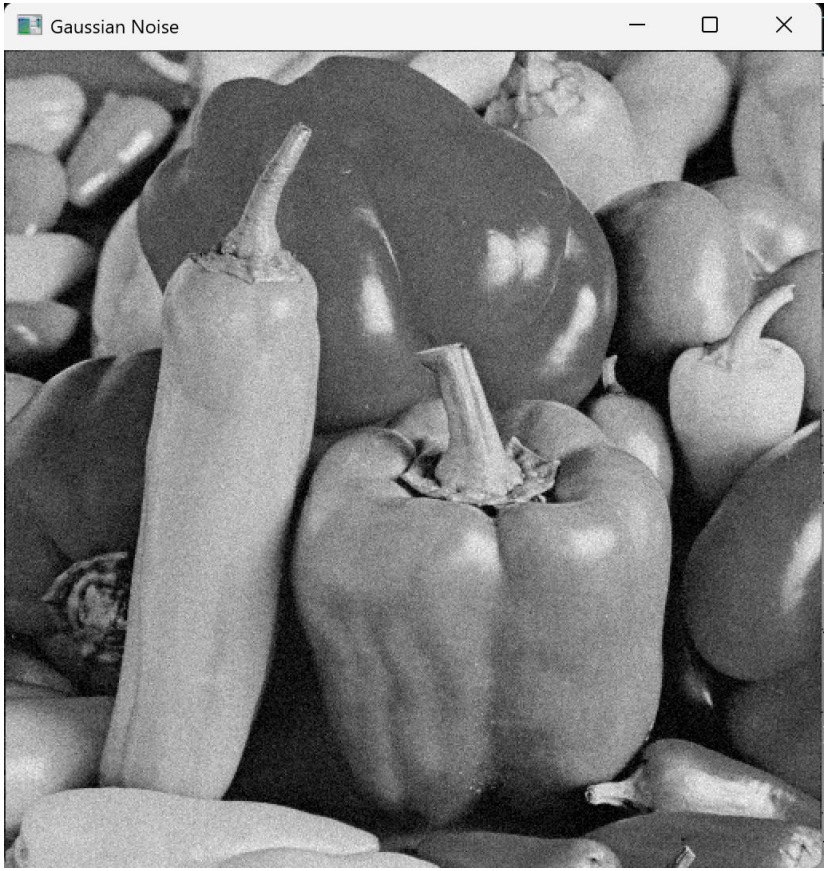
    # 5. Print MSE values
    print("=== Mean Squared Error (MSE) ===")
    print(f"Gaussian Noise       : {mse_gaussian_noisy:.2f}")
    print(f"Salt & Pepper Noise  : {mse_sp_noisy:.2f}")
    print(f"Avg Filter (Gauss)   : {mse_avg_gaussian:.2f}")
    print(f"Median Filter (S&P)  : {mse_med_sp:.2f}")

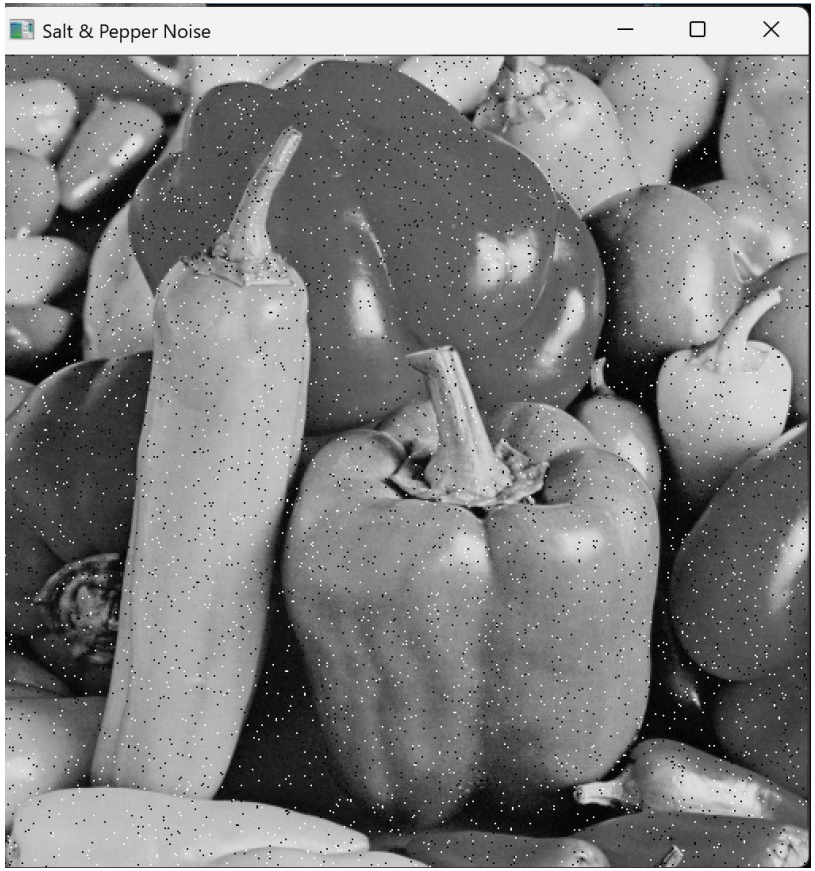
```

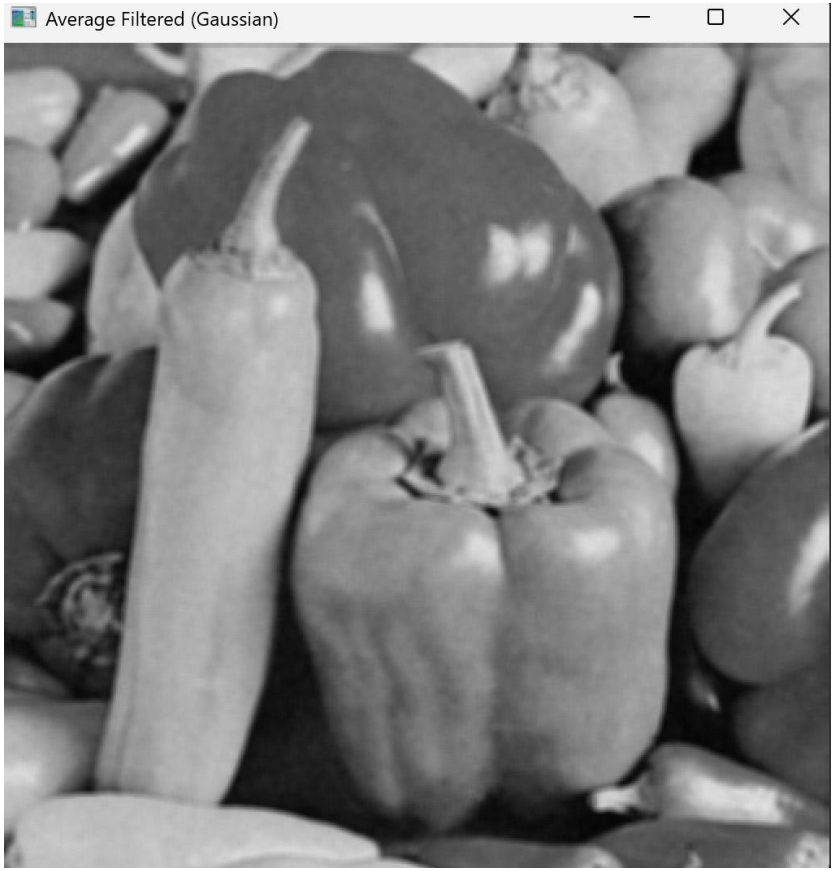


```
71
72      # 6. Display each image individually
73      cv2.imshow("Original (Grayscale)", original)
74      cv2.imshow("Gaussian Noise", noisy_gaussian)
75      cv2.imshow("Salt & Pepper Noise", noisy_sp)
76      cv2.imshow("Average Filtered (Gaussian)", avg_gaussian)
77      cv2.imshow("Median Filtered (S&P)", med_sp)
78
79      cv2.waitKey(0)
80      cv2.destroyAllWindows()
81
82  if __name__ == "__main__":
83      main()
84
```


Original (Grayscale)


Gaussian Noise


Salt & Pepper Noise


Average Filtered (Gaussian)


Median Filtered (S&P)

The **Mean Squared Error (MSE)**

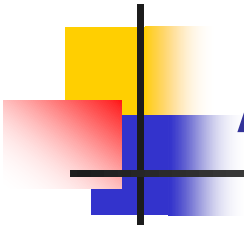
MSE between two images is calculated by:

1. Subtracting the corresponding pixel values in the two images.
2. Squaring these differences (to ensure they are non-negative).
3. Averaging the squared differences across all pixels.

MSE result

```
=== Mean Squared Error (MSE) ===
Gaussian Noise        : 148.49
Salt & Pepper Noise   : 385.50
Avg Filter (Gauss)    : 102.06
Median Filter (S&P)   : 39.05
```

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(I_1(i) - I_2(i)\right)^2$$



Amend the code

We did not apply both filters to both noisy images. Instead, we used the most commonly recommended pairing:

- Gaussian noise is often handled reasonably well by a mean (average) filter.
- Salt & Pepper noise is typically better removed by a median filter.

Amend the code

- If you wanted to also apply the median filter to the Gaussian-noisy image (or the average filter to the Salt & Pepper image), you'd just add two extra lines, for example:

```
med_gaussian = cv2.medianBlur(noisy_gaussian, 5)
avg_sp = cv2.blur(noisy_sp, (5,5))
```